

CSE208 - Data Structures and Algorithm Sessional

Online on Hashing

Section: B

Duration: 30 minutes

Task

Extend your existing **chaining-based hash table** to support **priority-weighted ordering** within each chain.

Specification

1. At first change your **Hash1** function to the following **position-weighted** hash (note: this is a different hash function from the other class assignments on hashing):

$$\text{Hash1}(k) = \left(\sum_{i=1}^{|k|} i \times \text{ASCII}(k_i) \right) \bmod N$$

where k_i is the i -th character of k (1-indexed), so each character's contribution is weighted by its position in the word. Use $N = 13$ (fixed — **do not resize** for this exercise).

2. **Weight function.** Each key's weight must be **computed** from the word itself using the following function:

$$\text{weight}(k) = \frac{(\text{ASCII}(k_1) + \text{ASCII}(k_{|k|})) \bmod 100}{100}$$

i.e., take the ASCII codes of the **first and last character** of the word, sum them, reduce modulo 100, and divide by 100 to land in $[0, 1]$.

3. **Insertion rule.** When inserting into a chain (linked list at a given slot), the new node must be placed such that the chain remains **sorted in descending order of weight** — the highest-weight key is always at the head of the chain, and weights decrease (or stay equal) moving toward the tail.

- Duplicate words are still discarded as before (check for existing key before insertion, regardless of weight).
- If two keys have equal weight, insertion order is used as a tiebreak — the later one goes after the former one.

4. Run for the given Input in the next section.

Input

Insert the following 12 words, **in this exact order**, using sequence numbers 1–12 as values:

cat, dog, bat, rat, sun, fun, run, top, pot, opt, art, tar

Output

The hash value and weight of the given words should match the below table.

Word	Hash1	weight	Word	Hash1	weight
cat	4	0.15	top	11	0.28
dog	7	0.03	pot	6	0.28
bat	3	0.14	opt	7	0.27
rat	6	0.30	art	10	0.13
sun	3	0.25	tar	2	0.30
fun	3	0.12	run	2	0.24

Using the sorted-chaining rule, report the final chain contents (head to tail) at every non-empty bucket. our code should generate the following output.

Bucket	Chain (head → tail)
2	tar (0.30) → run (0.24)
3	sun (0.25) → bat (0.14) → fun (0.12)
4	cat (0.15)
6	rat (0.30) → pot (0.28)
7	opt (0.27) → dog (0.03)
10	art (0.13)
11	top (0.28)
0,1,5,8,9,12	(empty)